

Chapter 3. Overview of the JVM

There is no doubt that Java is one of the largest technology platforms on the planet—the best available estimate is of over 10+ million developers working with Java.

The design of the JVM is high level, with the aim of removing complexity for developers. Key aspects, such as garbage collection and execution optimization are kept under the control of the JVM on behalf of developers. The fact that Java is consciously aimed at mainstream developers leads to a situation in which many developers do not need to know about the low-level intricacies of the platform they work with daily. As a result, developers may not meet these internal aspects very frequently—but only when an issue arises, such as a customer complaining about a performance problem.

For developers who are interested in performance, however, it is important to understand the basics of the JVM technology stack. Understanding JVM technology enables developers to write better software and provides the theoretical background required for investigating performance-related issues.

This chapter introduces how the JVM executes Java to provide a basis for deeper exploration of these topics later in the book. In particular, [Chapter 6](#) has an in-depth treatment of bytecode, which is complementary to the discussion here.

We suggest that you read through this chapter then come back to it for a second pass after you have read [Chapter 6](#).

Interpreting and Classloading

According to the specification that defines the Java virtual machine (usually called the VM spec), the JVM is a stack-based interpreted machine. This means that rather than having registers (like a physical hardware

CPU), it uses an execution stack of partial results and performs calculations by operating on the top value (or values) of that stack.

If you're not familiar with how interpreters work, then you can think of the basic behavior of the JVM interpreter as essentially “a `switch` inside a `while` loop.” The interpreter processes each opcode of the program independently of the last and uses the evaluation stack to hold the results of computations and as intermediate results.

NOTE

As we will see when we delve into the internals of the Oracle/OpenJDK VM (HotSpot), the situation for real production-grade Java interpreters is more complex, but *switch-inside-while* using a stack interpreter is an acceptable mental model for the moment.

When we launch our application using the `java HelloWorld` command, the operating system starts the virtual machine process (the `java` binary). This sets up the Java virtual environment and initializes the interpreter that will actually execute the user code in the `HelloWorld.class` file.

The entry point into the application will be the `main()` method of `HelloWorld.class`. To hand over control to this class, it must be loaded by the virtual machine before execution can begin.

To achieve this, the Java class loading mechanism is used. When a new Java process is initializing, a chain of class loaders is used. The initial loader is known as the Bootstrap class loader (historically also known as the “Primordial class loader”), and it loads classes in the core Java runtime. The main point of the Bootstrap class loader is to get a minimal set of classes (which includes essentials such as `java.lang.Object`, `Class`, and `ClassLoader`) loaded to allow other class loaders to bring up the rest of the system.

At this point, it is also instructive to discuss a little bit of how the Java Platform Module System (sometimes referred to as JPMS) has changed the picture of application startup somewhat. First of all, from Java 9 onward, all JVMs are modular--there is no “compatibility” or “classic” mode that restores the Java 8 monolithic JVM runtime.

This means that during startup a module graph is always constructed—even if the application itself is nonmodular. This must be a directed acyclic graph (DAG), and it is a fatal startup error if the application’s module metadata attempts to construct a module graph that contains a cycle.

The module graph has various advantages, including:

- Only required modules are loaded.
- Inter-module metadata can be confirmed to be good at startup time.

The module graph has a main module, which is where the `entrypoint` class lives. If the application has not yet been fully modularized, then it will have both a `modulepath` and a `classpath`, and the application code may be in the `UNNAMED` module.

NOTE

Full details of the modules system are outside the scope of this book. An expanded treatment can be found in [*Java in a Nutshell, 8th Edition*](#) by Benjamin J. Evans, Jason Clark, and David Flanagan (O’Reilly) or a more in-depth reference, such as [*Java 9 Modularity*](#) by Sander Mak and Paul Bakker (O’Reilly).

In practice, the work of the Bootstrap class loader involves loading `java.base` and some other supporting modules (including some perhaps-surprising entries—e.g., `java.security.sasl` and `java.datatransfer`).

Java models class loaders as objects within its own runtime and type system, so there needs to be some way to bring an initial set of classes into existence. Otherwise, there would be a circularity problem in defining what a class loader is.

The Bootstrap class loader does not verify the classes it loads (largely to improve startup performance), and it relies on the boot classpath being secure. Anything loaded by the Bootstrap class loader is granted full security permissions, so this group of modules is kept as restricted as possible.

NOTE

Legacy versions of Java up to and including 8 used a monolithic runtime, and the Bootstrap class loader loaded the contents of `rt.jar`.

The rest of the base system (i.e., the equivalent of the rest of the old `rt.jar` used in version 8 and earlier) is loaded by the *Platform class loader*, which is available via the method `ClassLoader::getPlatformClassLoader`. It has the Bootstrap class loader as its parent, as the old Extension class loader has been removed.

In the new modular implementations of Java, far less code is required to bootstrap a Java process, and accordingly, as much JDK code (now represented as modules) as possible has been moved out of the scope of the bootstrap loader and into the platform loader instead.

Finally, the Application class loader is created; it is responsible for loading user classes from the defined classpath. Some texts, unfortunately, refer to this as the “System” class loader. This term should be avoided, for the simple reason that it doesn’t load the system classes (the Bootstrap and Platform class loaders do). The Application class loader is encountered extremely frequently, and it has the Platform loader as its parent.

Java loads in dependencies on new classes when they are first encountered during the execution of the program. If a class loader fails to find a class, the behavior is usually to delegate the lookup to the parent. If the chain of lookups reaches the Bootstrap class loader and it isn’t found, a `ClassNotFoundException` will be thrown. It is important that developers use a build process that effectively compiles with the exact same classpath that will be used in production, as this helps to mitigate this potential issue.

Normally, Java loads a class only once, and a `Class` object is created to represent the class in the runtime environment. However, it is important to realize that under some circumstances the same class can be loaded twice by different class loaders. As a result, a class in the system is identified by the class loader used to load it as well as the fully qualified class name (which includes the package name).

NOTE

Some execution contexts, such as application servers (e.g., Tomcat or JBoss EAP) by design display this behavior when multiple tenant applications are present in the server. This enables different tenants to have different versions of the classes they need.

It is also the case that some tools (e.g., Java agents) can potentially reload and retransform classes as part of bytecode weaving--and such tools are often used in monitoring and observability.

Executing Bytecode

It is important to appreciate that Java source code goes through a significant number of transformations before execution. The first is the compilation step using the Java compiler `javac`, often invoked as part of a larger build process.

The job of `javac` is to convert Java code into `.class` files that contain bytecode. It achieves this by doing a fairly straightforward translation of the Java source code, as shown in [Figure 3-1](#). The diagram clearly shows that the creation of class files is not the whole story—and much of the JVM's power comes from what happens at runtime.

Accordingly, very few optimizations are done during compilation by `javac`, and the resulting bytecode is still quite readable and recognizable as Java code when viewed in a disassembly tool, such as the standard `javap`.

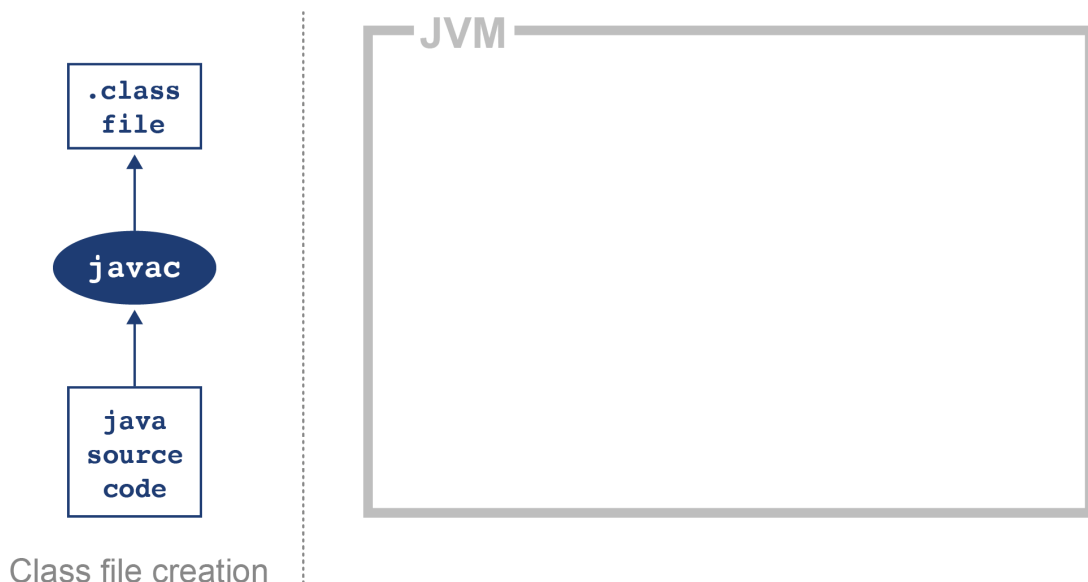


Figure 3-1. Java class file compilation

Bytecode is an *intermediate representation* that is not tied to a specific machine architecture. Decoupling from the machine architecture provides portability, meaning already developed (or compiled) software can run on any platform supported by the JVM and provides an abstraction from

the Java language. This provides our first important insight into the way the JVM executes code.

The Java language and the Java virtual machine are now independent to a degree, so the J in JVM is potentially a little misleading, as the JVM can execute any JVM language that can produce a valid class file. In fact, [Figure 3-1](#) could just as easily show the Kotlin compiler `kotlinc` generating bytecode for execution on the JVM.

Regardless of the source code compiler used, the resulting class file has a very well-defined structure specified by the VM spec ([Table 3-1](#)). Any class that is loaded by the JVM will be verified to conform to the expected format before being allowed to run.

Table 3-1. Anatomy of a class file

Component	Description
Magic number	0xCAFEFEBABE
Version of class file format	The minor and major versions of the class file
Constant pool	The pool of constants for the class
Access flags	Whether the class is abstract, static, and so on
This class	The name of the current class
Superclass	The name of the superclass
Interfaces	Any interfaces in the class
Fields	Any fields in the class
Methods	Any methods in the class
Attributes	Any attributes of the class (e.g., name of the source file)

Every class file starts with the magic number `0xCAFEFEBABE`, the first 4 bytes in hexadecimal serving to denote conformance to the class file format. The following 4 bytes represent the minor and major versions used to compile the class file, and these are checked to ensure that the version of the JVM is not of a lower version than the one used to compile the class file. The major and minor version are checked by the class loader to ensure compatibility; if these are not compatible, an

`UnsupportedClassVersionError` will be thrown at runtime, indicating the runtime is a lower version than the compiled class file.

NOTE

Magic numbers provide a way for Unix environments to identify the type of a file (whereas Windows will typically use the file extension). For this reason, they are difficult to change once decided upon. Unfortunately, this means that Java is stuck using the rather embarrassing and sexist `0xCAFEBABE` for the foreseeable future.

The constant pool holds constant values in code: for example, names of classes, interfaces, and fields. When the JVM executes code, the constant pool table is used to refer to values rather than having to rely on the precise layout of memory structures at runtime.

Access flags are used to determine the modifiers applied to the class. The first part of the flag block identifies general properties, such as whether a class is public, followed by whether it is final and thus cannot be subclassed. The flags also determine whether the class file represents an interface or an abstract class. The final part of the flag block indicates whether the class file represents a synthetic class (i.e., one that is not present in source code), an annotation type, or an enum.

The `this` class, superclass, and interface entries are indexes into the constant pool to identify the type hierarchy belonging to the class. Fields and methods define a signature-like structure, including the modifiers that apply to the field or method. A set of attributes is then used to represent structured items for more complicated and non-fixed-size structures. For example, methods use the `Code` attribute to represent the bytecode associated with that particular method.

[Figure 3-2](#) provides a mnemonic for remembering the structure.



My	Very	Cute	Animal	Turns	Savage	In	Full	Moon	Areas
M	V	C	A	T	S	I	F	M	A
Magic	Version	Constant	Access	This	Super	Interfaces	Fields	Methods	Attributes

Figure 3-2. Mnemonic for class file structure

In this very simple code example, it is possible to observe the effect of running `javac`:

```
public class HelloWorld {
    public static void main(String[] args) {
        for (int i = 0; i < 10; i++) {
            System.out.println("Hello World");
        }
    }
}
```

Java ships with a class file disassembler called `javap`, allowing inspection of `.class` files. Taking the `HelloWorld` class file and running `javap -c HelloWorld` gives the following output:

```
public class HelloWorld {
    public HelloWorld();
        Code:
            0: aload_0
            1: invokespecial #1    // Method java/lang/Object."<init>":()V
            4: return

    public static void main(java.lang.String[]);
        Code:
            0: iconst_0
            1: istore_1
            2: iload_1
            3: bipush      10
            5: if_icmpge   22
            8: getstatic   #2    // Field java/lang/System.out ...
           11: ldc         #3    // String Hello World
```



```

13: invokevirtual #4      // Method java/io/PrintStream.println .
16: iinc           1, 1
19: goto          2
22: return

}

```

This layout describes the bytecode for the file *HelloWorld.class*. For more detail, `javap` also has a `-v` option that provides the full class file header information and constant pool details. The class file contains two methods, although only the single `main()` method was supplied in the source file; this is the result of `javac` automatically adding a default constructor to the class.

The first instruction executed in the constructor is `aload_0`, which places the `this` reference onto the first position in the stack. The `invokespecial` command is then called, which invokes an instance method that has specific handling for calling superconstructors and creating objects. In the default constructor, the `invoke` matches the default constructor for `Object`, as an override was not supplied.

NOTE

[Opcodes in the JVM](#) are concise and represent the type, the operation, and the interaction among local variables, the constant pool, and the stack. See [Chapter 6](#) for more details.

Moving on to the `main()` method, `iconst_0` pushes the integer constant `0` onto the evaluation stack. `istore_1` stores this constant value into the local variable at offset 1 (represented as `i` in the loop). Local variable offsets start at 0, but for instance methods, the 0th entry is always `this`. The variable at offset 1 is then loaded back onto the stack, and the constant `10` is pushed for comparison using `if_icmpge` (“if integer compare greater or equal”). The test succeeds only if the current integer is `>= 10`.

For the first 10 iterations, this comparison test fails, so we continue to instruction 8. Here the static method from `System.out` is resolved, followed by the loading of the “Hello World” string from the constant pool. The next `invoke`, `invokevirtual`, invokes an instance method based on the class. The integer is then incremented, and `goto` is called to loop back to instruction 2.

This process continues until the `if_icmpge` comparison eventually succeeds (when the loop variable is ≥ 10); on that iteration of the loop, control passes to instruction 22, and the method returns.

Introducing HotSpot

In April 1999, Sun introduced HotSpot—one of the biggest-ever changes (in terms of performance) to the dominant Java implementation. HotSpot is a virtual machine that has evolved to enable performance that is comparable to (or better than) languages such as C and C++ (see [Figure 3-3](#)). To explain how this is possible, let’s delve a little deeper into the design of languages intended for application development.

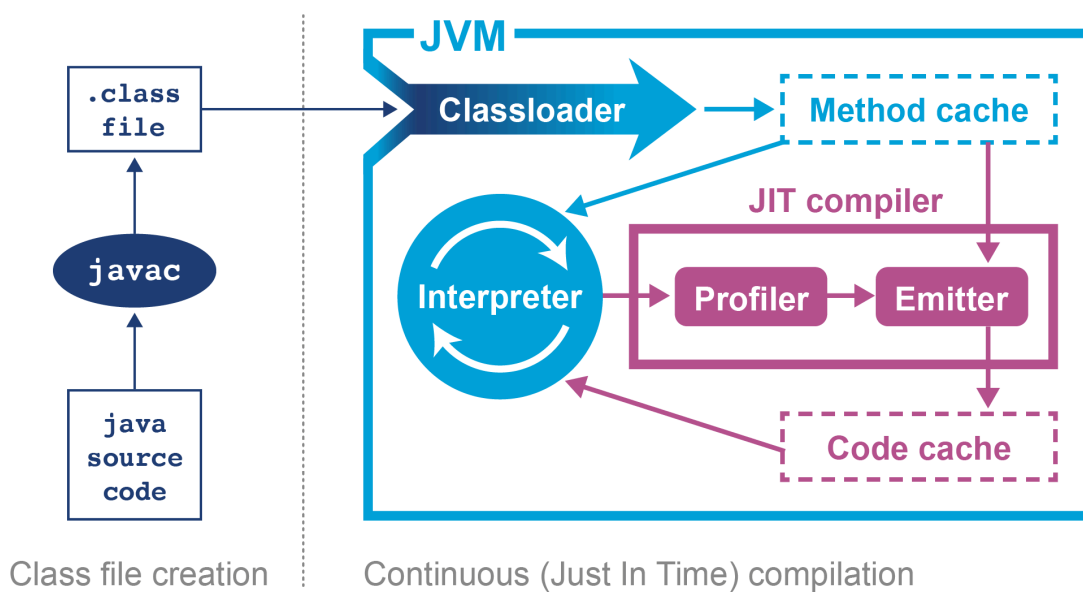


Figure 3-3. The HotSpot JVM

Language and platform design frequently involves making decisions and tradeoffs between desired capabilities. In this case, the division is between languages that stay “close to the metal” and rely on ideas such as “zero-cost abstractions,” and languages that favor developer productivity and “getting things done” over strict low-level control.

*In general, C++ implementations obey the zero-overhead principle:
What you don’t use, you don’t pay for. And further: What you do use,
you couldn’t hand code any better.*¹

—Bjarne Stroustrup

The zero-overhead principle sounds great in theory, but it requires all users of the language to deal with the low-level reality of how operating

systems and computers actually work. This is a significant extra cognitive burden that is placed upon developers who may not care about raw performance as a primary goal.

Not only that, but it also requires the source code to be compiled to platform-specific machine code at build time—usually called *ahead-of-time* (AOT) compilation. This is because alternative execution models such as interpreters, virtual machines, and portability layers all are most definitely not zero-overhead.

The phrase “what you do use, you couldn’t hand code any better” also has a sting in its tail. It implies a number of things, but most important for our purposes is that a developer is not able to produce better code than an automated system (such as a compiler).

Java has never subscribed to the zero-overhead abstraction philosophy. Instead, the approach taken by the HotSpot virtual machine is to analyze the runtime behavior of your program and intelligently apply optimizations where they will benefit performance the most. The goal of the HotSpot VM is to allow you to write idiomatic Java and follow good design principles rather than contort your program to fit the JVM.

Introducing Just-in-Time Compilation

Java programs begin their execution in the bytecode interpreter, where instructions are performed on a virtualized stack machine. This abstraction from the CPU gives the benefit of class file portability, but to get maximum performance, your program must make optimal use of its native features.

HotSpot achieves this by compiling units of your program from interpreted bytecode into native code, which then executes directly, without requiring the overhead of the abstractions of the interpreter. The units of compilation in the HotSpot VM are the method and the loop. This is known as *just-in-time* (JIT) compilation.

JIT compilation works by monitoring the application while it is running in interpreted mode and observing the parts of code that are most frequently executed. During this analysis process, programmatic trace information is captured that allows for more sophisticated optimization. Once

execution of a particular method passes a threshold, the profiler will look to compile and optimize that particular section of code.

There are many advantages to the JIT approach to compilation, but one of the main ones is that it bases compiler optimization decisions on trace information that is collected while methods are being interpreted. This information enables HotSpot to make more informed optimizations if the method is eligible for compilation.

NOTE

Some JIT compilers also have the capability to re-JIT if a better optimization becomes apparent later on during execution. This includes some of HotSpot's compilers.

Not only that, but HotSpot has had hundreds of engineering years (or more) of development attributed to it, and new optimizations and benefits are added with almost every new release. This means that all Java applications benefit from the latest HotSpot performance optimizations in the VM without even needing to be recompiled.

TIP

After being translated from Java source to bytecode and now going through another step of (JIT) compilation, the code actually being executed has changed very significantly from the source code as written. This is a key insight, and it will drive our approach to dealing with performance-related investigations. JIT-compiled code executing on the JVM may well look nothing like the original Java source code.

The general picture is that languages like C++ and Rust tend to have more predictable performance but at the cost of forcing a lot of low-level complexity onto the user.

Note also that “more predictable” does not necessarily mean “better.” AOT compilers produce code that may have to run across a broad class of processors—and usually are not able to assume that specific processor features are available.

Environments that use profile-guided optimization (PGO), such as Java, have the potential to use runtime information in ways that are simply impossible to most AOT platforms. This can offer improvements to performance, such as dynamic inlining and optimizing away virtual calls. HotSpot can even detect the precise CPU type it is running on at VM start-up and can use this information to enable optimizations designed for specific processor features if available.

TIP

The technique of detecting precise processor capabilities is known as *JVM intrinsics* and is not to be confused with the intrinsic locks introduced by the `synchronized` keyword.

A full discussion of PGO and JIT compilation can be found in [Chapter 6](#).

The sophisticated approach that HotSpot takes is a great benefit to the majority of ordinary developers, but this tradeoff (to abandon zero-overhead abstractions) means that in the specific case of high-performance Java applications, the developer must be very careful to avoid “common sense” reasoning and overly simplistic mental models of how Java applications actually execute.

NOTE

Once again, analyzing the performance of small sections of Java code (*micro-benchmarks*) is usually much harder than analyzing entire applications, and is a very specialized task that the majority of developers should not undertake.

HotSpot’s compilation subsystem is one of the two most important subsystems that the virtual machine provides. The other is automatic memory management, which has been one of the major selling points of Java since the early years.

JVM Memory Management

In languages such as C, C++, and Objective-C the programmer is responsible for managing the allocation and release of memory. The benefits of managing memory and lifetime of objects yourself are more determinis-

tic performance and the ability to tie resource lifetime to the creation and deletion of objects. However, these benefits come at a huge cost—for correctness, developers must be able to accurately account for memory.

Unfortunately, decades of practical experience showed that many developers have a poor understanding of patterns for memory management. Later versions of C++ and Objective-C have improved this somewhat by using smart pointer idioms in the standard library. However, at the time Java was created, poor memory management was a major cause of application errors. This led to concern among developers and managers about the amount of time spent dealing with language features rather than delivering value for the business.

Java looked to help resolve the problem by introducing automatically managed heap memory using a process known as *garbage collection* (GC). Simply put, garbage collection is a nondeterministic process that triggers the recovery and reuse of no-longer-needed memory when the JVM requires more memory for allocation.

GC comes at a cost: when it runs, it traditionally *stopped the world*, which means while GC is in progress, the application pauses. Usually these pause times are incredibly short, but as an application is put under pressure, these times can increase.

Having said that, the JVM's garbage collection in 2024 is best in class and is far more sophisticated than the introductory algorithm that is often taught in computer science undergraduate courses. For example, stopping the world is much less necessary and intrusive in modern algorithms, as we will see later.

Removing the concern of memory management from the developer provides the benefits of improving the correctness of an application. Garbage collection impacts performance and is an important subsystem in the JVM. The impact has the potential to be both positive and negative to the running application.

Garbage collection is a major topic within Java performance optimization, so we will devote Chapters [4](#) and [5](#) to the details of Java GC.

Threading and the Java Memory Model

One of the major advances that Java brought in with its first version was built-in support for multithreaded programming. The Java platform allows the developer to create new threads of execution. For example, in Java 8 syntax:

```
Thread t = new Thread(() -> {System.out.println("Hello World!");});  
t.start();
```

Not only that, but basically all production JVMs are multithreaded—and this means that all Java programs are inherently multithreaded, as they execute as part of a JVM process.

This fact produces additional, irreducible complexity in the behavior of Java programs, and it makes the work of the performance analyst harder. However, it allows the JVM to take advantage of all available cores, which provides all sorts of performance benefits to the Java developer.

The relationship between Java’s conception of a thread (an “application thread”) and the operating system’s view of a thread (a “platform thread”) has an interesting history. In the very earliest days of the platform, a sharp distinction was made between the two concepts, and application threads were *remapped* or *multiplexed* onto a pool of platform threads—e.g., in the Solaris *M:N*, or the Linux *green threads* models.

However, this approach proved not to provide an acceptable performance profile and added needless complexity. As a result, in most mainstream JVM implementations, this model was replaced with a simpler one—each Java application thread corresponding precisely to a dedicated platform thread.

This is not the end of the story, however.

In the 20+ years since the “app thread == platform thread” transition, applications have grown and scaled massively—and so has the number of threads (or, more generally, *execution contexts*) that an application might want to create. This has led to the “thread bottleneck” problem, and solving it has been the focus of a major research project within OpenJDK (Project Loom).

The result is *virtual threads*, a new form of thread available only in Java 21+, which can be used efficiently for certain tasks—especially those performing network I/O.

Programmers must explicitly choose to create a thread as virtual—otherwise they are platform threads and retain the same behavior as before (so the semantics of all existing Java programs are preserved when run on a JVM with virtual thread capability).

NOTE

It is safe to assume that every platform thread (or any thread, before Java 21) is backed by a unique OS thread that is created when the `start()` method is called on the corresponding `Thread` object.

Virtual threads are Java’s take on an idea that can be found in various other modern languages—for example, Go programmers may regard a Java virtual thread as being broadly similar to a goroutine. We will discuss virtual threads in more detail in [Chapter 13](#).

We should also briefly discuss Java’s approach to handling data in a multithreaded program. It dates from the late 1990s and has these fundamental design principles:

- All threads in a Java process share a single, common garbage-collected heap.
- Any object created by one thread can be accessed by any other thread that has a reference to the object.
- Objects are mutable by default; that is, the values held in object fields can be changed unless the programmer explicitly uses the `final` keyword to mark them as immutable.

The Java Memory Model (JMM) is a formal model of memory that explains how different threads of execution see the changing values held in objects. That is, if threads A and B both have references to object `obj`, and thread A alters it, what happens to the value observed in thread B?

This seemingly simple question is actually more complicated than it seems, because the operating system scheduler (which we will meet in [Chapter 7](#)) can forcibly evict platform threads from CPU cores. This can lead to another thread starting to execute and accessing an object before

the original thread had finished processing it, and potentially seeing the object in a prior or even invalid state.

The only defense the core of Java provides against this potential object damage during concurrent code execution is the mutual exclusion lock, and this can be very complex to use in real applications. [Chapter 13](#) contains a detailed look at how the JMM works and the practicalities of working with threads and locks.

Monitoring and Tooling for the JVM

The JVM is a mature execution platform, and it provides a number of technology alternatives for instrumentation, monitoring, and observability of running applications. The main technologies available for these types of tools for JVM applications are:

- Java Management Extensions (JMX)
- Java agents
- The JVM Tool Interface (JVMTI)
- The Serviceability Agent (SA)

JMX is a general-purpose technology for controlling and monitoring JVMs and the applications running on them. It provides the ability to change parameters and call methods in a general way from a client application. A full treatment of how this is implemented is, unfortunately, outside the scope of this book. However, JMX (and its associated network transport, *Remote Method Invocation* or RMI) is a fundamental aspect of the management capabilities of the JVM.

A Java agent is a tooling component that uses the interfaces in `java.lang.instrument` to modify the bytecode of methods as classes are loaded. The modification of bytecode allows instrumentation logic, such as method timing or distributed tracing (see [Chapter 10](#) for more details), to be added to any application, even one that has not been written with any support for those concerns.

This is an extremely powerful technique, and installing an agent changes the standard application lifecycle that we met in the last section. To be installed, an agent must be packaged as a JAR and provided via a startup flag to the JVM:

```
-javaagent:<path-to-agent-jar>=<options>
```

The agent JAR must contain a manifest file, `META-INF/MANIFEST.MF`, and it must include the attribute `Premain-Class`.

This attribute contains the name of the agent class, which must implement a public static `premain()` method that acts as the registration hook for the Java agent. This method will run on the main application thread *before* the `main()` method of the application (hence the name). Note that the `premain` method must exit, or the main application will not start.

Bytecode transformation is the usual intent of an agent, and this is done by creating and registering bytecode transformers—objects that implement the `ClassFileTransformer` interface.

However, a Java agent is just Java code, so it can do anything that any other Java program can, i.e., it can contain arbitrary code to execute. This flexibility means that, for example, an agent can start additional threads that can persist for the entire life of the application, and can collect data for sending out of the application and into an external monitoring system.

NOTE

We will have a little more to say about JMX and agents in [Chapter 11](#), where we discuss their use in cloud observability tools.

If the Java instrumentation API is not sufficient, then the JVMTI may be used instead. This is a native interface of the JVM, so agents that use it must be written in a native compiled language—essentially, C or C++. It can be thought of as a communication interface that allows a native agent to monitor and be informed of events by the JVM. To install a native agent, provide a slightly different flag:

```
-agentlib:<agent-lib-name>=<options>
```

or:

`-agentpath:<path-to-agent>=<options>`

The requirement that JVMTI agents be written in native code means that these agents can be more difficult to write and debug. Programming errors in JVMTI agents can damage running applications and even crash the JVM.

Therefore, where possible, it is usually preferable to write a Java agent over JVMTI code. Agents are much easier to write, but some information is not available through the Java API, and JVMTI may be the only possibility available to access that data.

The final approach is the Serviceability Agent. This is a set of APIs and tools that can expose both Java objects and HotSpot data structures.

The SA does not require any code to be run in the target VM. Instead, the HotSpot SA uses primitives like symbol lookup and reading of process memory to implement debugging capability. The SA has the ability to debug live Java processes as well as core files (also called *crash dump files*).

One tool that should be introduced at this point is VisualVM, which is a graphical tool based on the [NetBeans platform](#). It used to ship as part of the JDK and was previously distributed in Oracle JDK versions 6–8 and in GraalVM versions 19–23.0.

However, it has since been moved out of the main distribution, so developers will have to download the binary separately from [the VisualVM website](#). After downloading, you will have to ensure that the `jvisualvm` binary is added to your path, or you may get an obsolete version from an old Java version.

TIP

`jvisualvm` is a replacement for the now obsolete `jconsole` tool from earlier Java versions. If you are still using `jconsole`, you should move to VisualVM (there is a compatibility plug-in to allow `jconsole` plug-ins to run inside VisualVM).

When VisualVM is started for the first time, it will calibrate the machine it is running on, so there should be no other applications running that

might affect the performance calibration.

When the desktop application is started for the first time, a screen similar to that shown in [Figure 3-4](#) is displayed (after a short calibration pause).

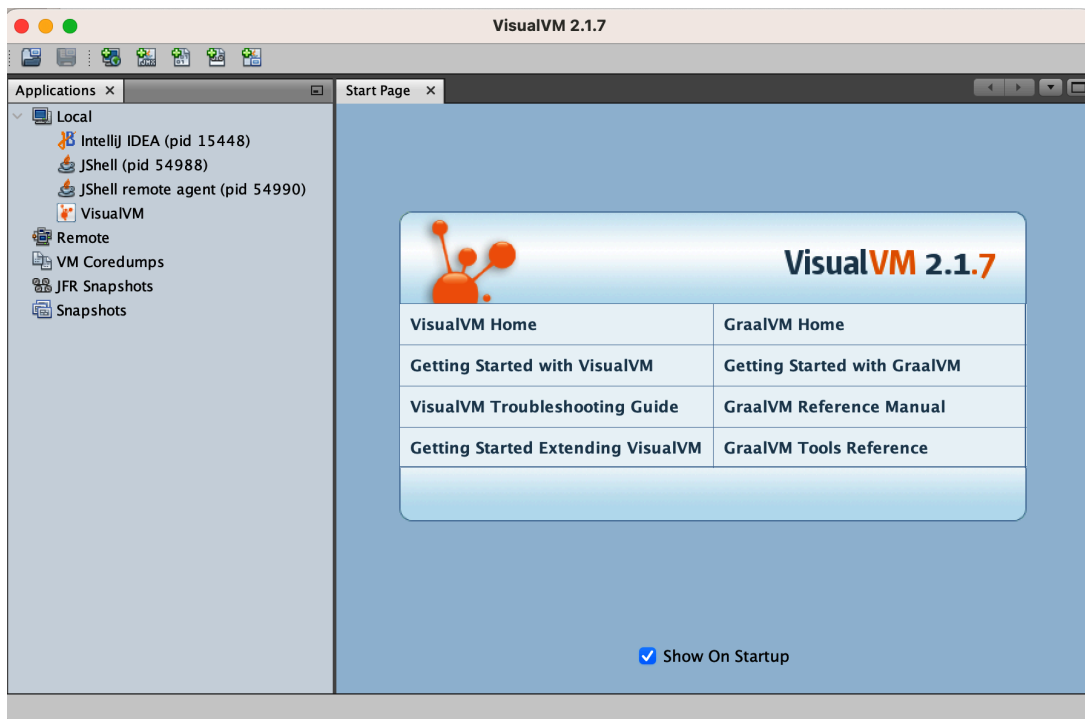


Figure 3-4. VisualVM startup screen

On the lefthand side are a choice of Local and Remote JVMs as well as snapshots and dump files. Note that VisualVM is, itself, a Java application.

Selecting a JVM from this sidebar gives the default view shown in [Figure 3-5](#).

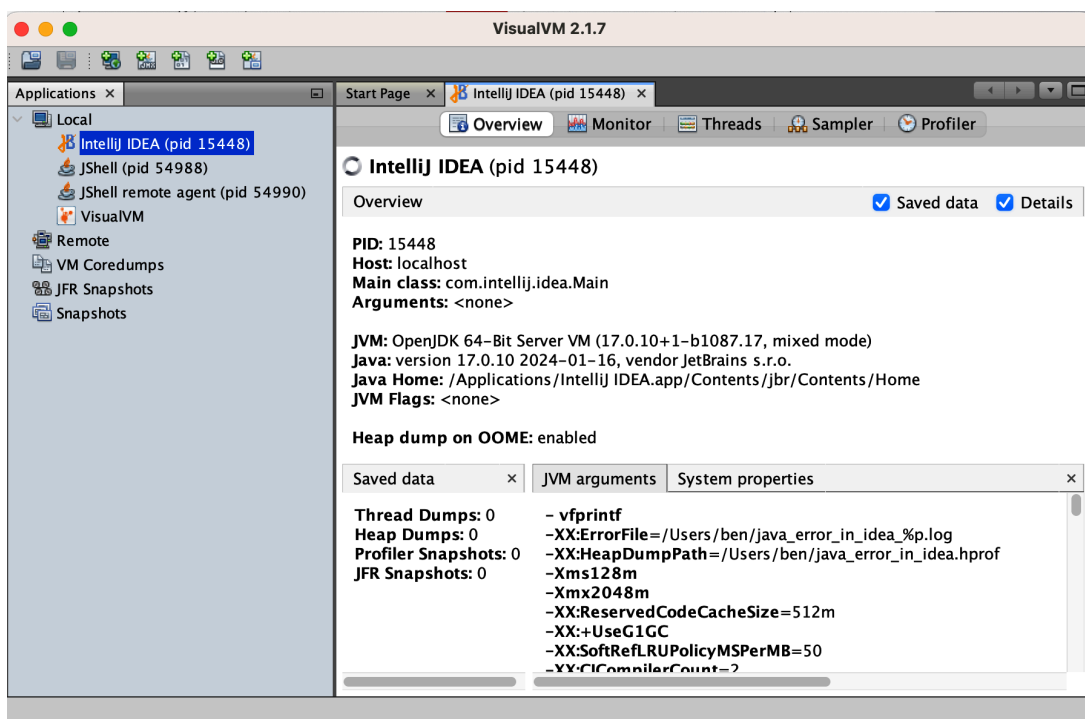


Figure 3-5. VisualVM default view

The most familiar view of VisualVM is the Monitor screen, which is similar to that shown in [Figure 3-6](#).

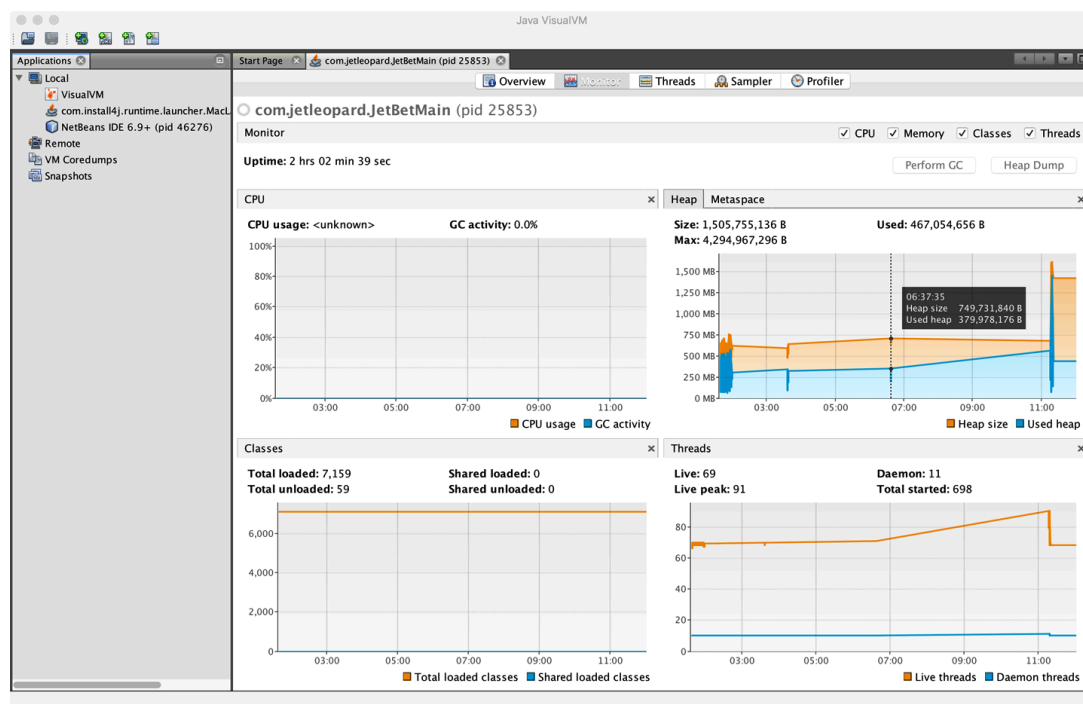


Figure 3-6. VisualVM Monitor screen

VisualVM is used for live monitoring of a running process, and it uses the JVM's *attach mechanism*. This works slightly differently depending on whether the process is local or remote.

Local processes are fairly straightforward. VisualVM lists them down the lefthand side of the screen. Double-clicking on one of them causes it to appear as a new tab in the righthand pane.

To connect to a remote process, the remote side must accept inbound connections (over JMX). For standard Java processes, this means `jstatd` must be running on the remote host (see the manual page for `jstatd` for more details).

NOTE

Many application servers and execution containers provide an equivalent capability to `jstatd` directly in the server. Such processes do not need a separate `jstatd` process so long as they are capable of port-forwarding JMX and RMI traffic.

To connect to a remote process, enter the hostname and a display name that will be used on the tab. The default port to connect to is 1099, but this

can be changed easily.

Out of the box, VisualVM presents the user with four tabs:

Overview

Provides a summary of information about your Java process. This includes the full flags that were passed in and all system properties. It also displays the exact Java version executing.

Monitor

This is the tab most similar to the legacy `jconsole` view. It shows high-level telemetry for the JVM, including CPU and heap usage. It also shows the number of classes loaded and unloaded, and an overview of the numbers of threads running.

Threads

Each thread in the running application is displayed with a timeline. This includes both application threads and VM threads. The state of each thread can be seen with a small amount of history. Thread dumps can also be generated if needed.

Sampler and Profiler

In these two tabs, simplified sampling of CPU and memory utilization can be accessed. This will be discussed more fully in [Chapter 12](#).

The plug-in architecture of VisualVM allows additional tools to be easily added to the core platform to augment the core functionality. These include plug-ins that allow interaction with JMX consoles and bridging to legacy JConsole, and a very useful garbage collection plug-in, VisualGC.

Java Implementations, Distributions, and Releases

In this section, we will briefly discuss the landscape of Java implementations and distributions as well as the Java release cycle.

This is an area that changes a lot over time—so this description is correct at the time of writing only. Since then, for example, vendors may have en-

tered (or exited) the business of making a Java distribution, or the release cycle may have changed. Caveat lector!

Many developers may only be familiar with the Java binaries produced by Oracle (Oracle JDK). However, as of 2024, we have quite a complex landscape, and it's important to understand the basic components of what makes up “Java.”

First, there's the source code that will be built into a binary. The source code required to build a Java implementation comes in two parts:

- Virtual machine source code
- Class library source code

The OpenJDK project, which can be found at [the OpenJDK website](#), is the project to develop the open source reference implementation of Java—which is licensed under the [GNU Public License version 2, with Classpath Exemption \(GPLv2+CE\)](#). The project is led and supported by Oracle—which provides a majority of the engineers who work on the OpenJDK codebase.

The critical point to understand about OpenJDK is that it provides *source code only*. This is true both for the VM (HotSpot) and for the class libraries.

The combination of HotSpot and the OpenJDK class libraries forms the basis of the vast majority of Java distributions used in today's production environments (including Oracle's). However, there are several other Java VMs that we will meet—and discuss briefly in this book—including Eclipse OpenJ9 and GraalVM. These VMs can also be combined with the OpenJDK class libraries to produce a complete Java implementation.

However, source code, by itself, is not all that useful to developers—it needs to be built into a binary distribution, tested, and optionally certified.

This is somewhat similar to the situation with Linux—the source code exists and is freely available, but in practice, virtually no one except those folks developing the next version work directly with source. Instead, developers consume a binary Linux distribution.

In the Java world there are a number of vendors who make distributions available, just as there are for Linux. Let's meet the vendors and take a quick look at their various offerings.

Choosing a Distribution

Developers and architects should consider carefully their choice of JVM vendor. Some large organizations—notably X (formerly Twitter) and Alibaba—even choose to maintain their own private (or semi-public) builds of OpenJDK, although the engineering effort required for this is beyond the reach of many companies.

With this in mind, the main factors that organizations typically care about are:

- Do I have to pay money to use this in production?
- How can I get any bugs I discover fixed?
- How do I get security patches?

To take these in turn:

A binary that has been built from OpenJDK source (which is GPLv2+CE-licensed) is free to use in production. This includes all binaries from Eclipse Adoptium, Red Hat, Amazon, and Microsoft, as well as binaries from lesser-known suppliers such as BellSoft. Some, but not all, of Oracle's binaries also fall into this category.

Next up, to get a bug fixed in OpenJDK, the discoverer may do one of two things: either buy a support contract and get the vendor to fix it, or ask an OpenJDK author to file a bug against the OpenJDK repo and then hope (or ask nicely) that someone fixes it for you. Or there's always the inevitable third option that all open source software provides—fix it yourself and then submit a patch.

The final point—about security updates—is slightly more subtle. First off, note that almost all changes to Java start off as commits to a public OpenJDK repository on GitHub. The exception to this is certain security fixes that have not yet been publicly disclosed.

When a fix is released and made public, there is a process by which the patch flows back into the various OpenJDK repos. The vendors will then be able to take that source code fix and build and release a binary that

contains it. However, there are some subtleties to this process, which is one reason why most Java shops prefer to remain on a long-term support (or LTS) version—we will have more to say about this in the section about Java versions.

Now that we’ve discussed the main criteria for choosing a distribution, let’s meet some of the main offerings available:

Oracle

Oracle’s Java (Oracle JDK is perhaps the most widely known implementation. It is essentially the OpenJDK codebase, relicensed under Oracle’s proprietary licenses with a few extremely minor differences (such as the inclusion of some additional components that are not available under an open source license). Oracle achieves this by having all contributors to OpenJDK sign a license agreement that permits dual licensing of their contribution to both the GPLv2+CE of OpenJDK and Oracle’s proprietary license.²

Eclipse Adoptium

This community-led project started life as AdoptOpenJDK, changing its name when it transitioned into the Eclipse Foundation. The members of the Adoptium project (from companies such as Red Hat, Google, Microsoft, and Azul) consist mostly of build and test engineers rather than development engineers (who implement new features and fix bugs). This is by design—many of Adoptium’s member companies also make major contributions to upstream OpenJDK development, but they do so under their own company names rather than Adoptium. The Adoptium project takes the OpenJDK source and builds fully tested binaries on multiple platforms. As a community project, Adoptium does not offer paid support, although member companies may choose to do so—for example, Red Hat does for some operating systems.

Red Hat

Red Hat is one of the the longest standing non-Oracle producers of Java binaries—as well as the second largest contributor to OpenJDK (behind Oracle). It produces builds and provides support for its operating systems—RHEL and Fedora—and Windows (for historical reasons). Red Hat also releases freely available container images based on its Universal Base Image (UBI) Linux system.

Amazon Corretto

Corretto is Amazon’s distribution of OpenJDK, and it is intended to run primarily on AWS cloud infrastructure. Amazon also provides builds for Mac, Windows, and Linux to provide a consistent developer experience and encourage developers to use their builds across all environments.

Microsoft OpenJDK

Microsoft has been producing binaries since May 2021 (OpenJDK 11.0.11) for Mac, Windows, and Linux. Just as for AWS, Microsoft’s distribution is largely intended to provide an easy on-ramp for developers who will be deploying on their Azure cloud infrastructure.

Azul Systems

Zulu is a free OpenJDK implementation provided by Azul Systems—which also offers paid support for its OpenJDK binaries. Azul also offers a high-performance proprietary JVM called “Azul Platform Prime” (previously known as Zing). Prime is not an OpenJDK distribution.

GraalVM

GraalVM is a relatively new addition to this list. Originally a research project at Oracle Labs, it has graduated to a fully productionized Java implementation (and much more too). GraalVM can operate in dynamic VM mode and includes an OpenJDK-based runtime—augmented with a JIT compiler that is written in Java. However, GraalVM is also capable of *native compilation* of Java—essentially AOT compilation. We will have more to say on this subject later in the book.

OpenJ9

OpenJ9 started life as IBM’s proprietary JVM (when it was just called J9) but was made open source in 2017 partway through its life (just like HotSpot). It is now built on top of an Eclipse open runtime project (OMR). It is fully compliant with Java certification. IBM Semeru Runtimes are zero-cost runtimes built with the OpenJDK class libraries and the Eclipse OpenJ9 JVM (which is Eclipse-licensed).

Android

Google’s Android project is sometimes thought of as being “based on Java.” However, the picture is actually a little more complicated. Android uses a cross compiler to convert class files to a different (*.dex*) file format. These *.dex* files are then executed by the Android Runtime (ART), which is not a JVM. In fact, Google now recommends the Kotlin language over Java for developing Android apps. As this technology stack is so far from the other examples, we won’t consider Android any further in this book.

Note that this list is not intended to be comprehensive—there are other distributions available as well.

The vast majority of the rest of this book focuses on the technology found in HotSpot. This means the material applies equally to Oracle’s Java and the distributions provided by Adoptium, Red Hat, Amazon, Microsoft, Azul Zulu, and all other OpenJDK-derived JVMs.

We also include some material related to Eclipse Open J9. This is intended to provide an awareness of alternatives rather than a definitive guide. Some readers may wish to explore these technologies more deeply, and they are encouraged to proceed by setting performance goals, and then measuring and comparing, in the usual manner.

Finally, before we discuss the Java release cycle, a word about the performance characteristics of the various OpenJDK distributions.

Teams occasionally ask questions about performance because they believe that certain distributions include different JIT or GC components that are not available in other OpenJDK-based distributions.

Let’s clear that up right now: all the OpenJDK distributions build from the same source, so there are no functional differences in the like-for-like versions—this is helped by the extremely robust test suite run on OpenJDK builds across all distributions.

In addition, there should be *no* systematic performance-related differences between the various HotSpot-based implementations when comparing like-for-like versions and build flag configurations. The only minor exception to this is that Oracle does not ship the Shenandoah garbage collector developed by Red Hat and Amazon and instead promotes its own ZGC collector.

NOTE

Some vendors choose very specific build flag combinations that are highly specific to their cloud environments, and some research indicates that these combinations *may* help for some subset of workloads, but this is far from clear-cut.

Once in a while, social media excitedly reports that significant performance differences have been found between some of the distributions. However, carrying out such tests in a sufficiently controlled environment is notoriously difficult—so any results should be treated with healthy skepticism unless they can be independently verified as statistically rigorous.

The Java Release Cycle

We can now complete the picture by briefly discussing the Java release cycle.

New feature development happens in the open—at a collection of GitHub repositories. Small to medium features and bug fixes are accepted as pull requests directly against the main branch in the [main OpenJDK repository](#). Larger features and major projects are frequently developed in forked repos and then migrated into mainline when ready.

Every six months, a new release of Java is cut from whatever is in main. Features that “miss the train” must wait for the next release—the six-month cadence and strict timescale has been maintained since September 2017. These releases are known as “feature releases,” and they are run by Oracle, in its role as steward of Java.

Oracle ceases to work on any given feature release as soon as the next feature release appears. However, an OpenJDK member of suitable standing and capability can offer to continue running the release after Oracle steps down. To date, this has only happened for certain releases—in practice Java 8, 11, 17, and 21, which are known as *update releases*.

The significance of these releases is that they match Oracle’s long-term support release concept. Technically, this is purely a construct of Oracle’s sales process—whereby Oracle customers who do not want to upgrade Java every six months have certain stable versions that Oracle will support them on.

In practice, the Java ecosystem has overwhelmingly rejected the official Oracle dogma of “upgrade your JDK every six months”—project teams and engineering managers simply have no appetite for it. Instead, teams upgrade from one LTS version to the next, and the update release projects (8u, 11u, 17u, and 21u) remain active, delivering security patches and a small number of bug fixes and backports. Oracle and the community work together to keep all these maintained code streams secure.

This is the final piece we need to answer the question of how to pick a Java distribution. If you want a zero-cost Java distribution that receives security patches and has a nonzero chance of security (and possibly bug) fixes, select your choice of OpenJDK vendor and stick to the LTS versions.

Any of: Adoptium, Red Hat, Amazon, and Microsoft is a fine choice—and so are some of the others. Depending on how and where you’re deploying your software (e.g., applications deploying in AWS may prefer Amazon’s Corretto distribution), you may have a reason to pick one of those over the others.

For a more in-depth guide to the various options and some of the licensing complexities, you can consult [Java Is Still Free](#). This document was written by the [Java Champions](#), an independent body of Java experts and leaders.

Summary

In this chapter, we have taken a quick tour through the overall anatomy of the JVM, including: compilation of bytecode, interpretation, JIT compilation to native code, memory management, threading, the lifecycle of a Java process monitoring, and finally, how Java is built and distributed.

It has been possible to touch on only some of the most important subjects, and virtually every topic mentioned here has a rich, full story behind it that will reward further investigation.

In [Chapter 4](#), we will begin our journey into garbage collection, starting with the basic concepts of mark-and-sweep and diving into the specifics, including some of the internal details of how HotSpot implements GC.

- ¹ Bjarne Stroustrup, “Abstraction and the C++ Machine Model,” *Lecture Notes in Computer Science*, vol. 3605 (Springer, 2005).
- ² The latter has changed multiple times, so linking to the currently latest version might not be helpful—it could be out of date by the time you read this.